

Mulham Al Kasir (Molham) — Training Sessions

Session 1 — Local State with useState

Learning Objectives

By the end of this session, the team should understand:

- What local state is.
- When to use `useState`.
- How pages in the project use it.

1. The Problem

The player selects an answer. The page needs to remember that selection. Without state, the page would forget it on every render.

2. Familiar Idea

In earlier sessions, we used `useState` for `currentFilter` and `searchText`. When the user typed or clicked, we updated state, and React re-rendered.

The same idea applies here.

3. The New Concept

Local state is a value React remembers between renders, for one component instance. `useState` returns a pair: the current value, and a function to change it.

```
```.tsx
const [selectedAnswer, setSelectedAnswer] = useState("");
```
```

Reading: `selectedAnswer` is the current value. Calling `setSelectedAnswer("brass")` schedules a re-render with the new value.

4. Tiny Example

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <button onClick={() => setCount(count + 1)}>Clicked {count} times</button>  
  );  
}
```

5. Project Application

File: `client/src/pages/GamePage.tsx`

```
function GameSession({ id }: { id: string }) {

  const [mystery, setMystery] = useState<Mystery | null>(null);

  const [clues, setClues] = useState<Clue[]>([]);

  const [selectedAnswer, setSelectedAnswer] = useState("");

  const [hint, setHint] = useState("");

  const [hintsRemaining, setHintsRemaining] = useState(0);

  const [showClues, setShowClues] = useState(false);

  const [message, setMessage] = useState("");

  const [feedbackTone, setFeedbackTone] = useState<"success" | "wrong">
("success");

  const [error, setError] = useState("");

  const [isLoading, setIsLoading] = useState(true);

  const [pending, setPending] = useState<"answer" | "hint" | null>(null);

  const [reload, setReload] = useState(0);

  ...

}
```

6. Code Walkthrough

State for the fetched data

```
const [mystery, setMystery] = useState<Mystery | null>(null);

const [clues, setClues] = useState<Clue[]>([]);
```

What it does: Holds the fetched mystery and clues.

Why it exists: The component needs the data across renders.

What you see: After fetch, the page renders the question and clues.

In Plain English: "Remember the mystery and the clues."

State for the player's input

```
const [selectedAnswer, setSelectedAnswer] = useState("");
```

What it does: Holds what the player currently has selected or typed.

Why it exists: The form is a controlled input.

What you see: The radio button or text input reflects this value.

State for UI toggles

```
const [showClues, setShowClues] = useState(false);
```

What it does: Tracks whether the clue panel is open.

Why it exists: The `CluePanel` needs an open/closed state.

What you see: Clicking "Show clues" toggles the panel.

State for async status

```
const [isLoading, setIsLoading] = useState(true);  
  
const [error, setError] = useState("");  
  
const [pending, setPending] = useState<"answer" | "hint" | null>(null);
```

What it does: Tracks what the page is doing right now.

Why it exists: The UI shows different things while loading, while submitting, and on error.

What you see: The submit button is disabled while pending; the loading message shows while loading.

The reload counter

```
const [reload, setReload] = useState(0);
```

What it does: A number that changes to trigger the load effect again.

Why it exists: After a correct answer, we want to re-fetch the server state.

What you see: Incrementing `reload` re-runs the effect.

7. Common Mistakes

WRONG: Mutating state directly.

```
selectedAnswer = "brass"; // wrong, React will not re-render
```

CORRECT:

```
setSelectedAnswer("brass");
```

WRONG: Putting too much in one state object.

```
const [state, setState] = useState({ mystery: null, clues: [], ... });
```

This makes every update harder to follow. The project uses separate `useState` calls for independent values.

WRONG: Using state for values that never change.

If a value is set once and never updated, `useRef` or a constant is better.

8. Important Decisions

- **Why separate `useState` calls?** Each value changes independently.
- **Why `null` for `mystery`?** The data is not loaded yet.
- **Why `""` for `selectedAnswer`?** An empty string means "nothing selected."
- **Why the reload counter?** A simple way to re-trigger an effect.

9. Project Flow

GamePage mounts

|

▼

useState initializes 12 pieces of state

|

▼

Player selects "brass" → setSelectedAnswer("brass")

|

▼

React re-renders with new value

|

▼

AnswerOptions shows "brass" as selected

10. Review Questions

1. What is local state?
2. When do you use `useState` ?
3. What does `setSelectedAnswer("brass")` do?
4. Why does the project use separate state calls?
5. What does the reload counter do?

11. Glossary

| Term | Meaning |
|------|---------|
|------|---------|

| State | A value React remembers between renders. |

| `useState` | A hook that returns a state value and a setter. |
| Re-render | React recalculating the UI from current values. |
| Controlled input | An input whose value comes from state. |

12. What You Should Be Able to Explain Now

- What state is and why it exists.
- How the project uses `useState`.
- Why separate calls for independent values.
- How the reload counter triggers re-fetching.

13. Not Yet

- Effects and cleanup — Session 2.
- Game flow handlers — Session 3.

Session 2 — Effects and Cleanup Guards

Learning Objectives

By the end of this session, the team should understand:

- What `useEffect` does.
- Why cleanup matters.
- How the project prevents stale responses.

1. The Problem

The page needs to fetch data when it mounts and when the id changes. Fetching takes time. If the user navigates away while the fetch is in flight, the response could try to update an unmounted component — a React warning.

Also, if the id changes from `1` to `2`, the old response for `1` could arrive after the new one and overwrite the correct data.

2. Familiar Idea

In earlier sessions, `useEffect` synchronized the component with an external system — the API.

The same idea applies here, but with a cleanup guard.

3. The New Concept

`useEffect` runs after render. It can return a cleanup function that runs before the next effect or on unmount.

The pattern:

```
useEffect(() => {
  let ignore = false;

  async function load() {
    const data = await fetchData();

    if (!ignore) setData(data);
  }

  load();

  return () => {
    ignore = true;
  };
}, [dependency]);
```

4. Tiny Example

```
useEffect(() => {
  let ignore = false;

  setTimeout(() => {
    if (!ignore) console.log("done");
  }, 1000);

  return () => {
    ignore = true;
  };
}, []);
```

5. Project Application

File: `client/src/pages/MysteriesPage.tsx`

```
useEffect(() => {
  let ignore = false;

  setIsLoading(true);

  setError("");

  async function load() {
    try {
      const data = await getMysteries();

      if (!ignore) setMysteries(data);
    } catch (error) {
      if (!ignore) {
        setError(
          error instanceof Error ? error.message : "Could not load mysteries.",
        );
      }
    } finally {
      if (!ignore) setIsLoading(false);
    }
  }

  void load();

  return () => {
    ignore = true;
  };
}, [reload]);
```

File: client/src/pages/GamePage.tsx

```
useEffect(() => {
  let ignore = false;

  setIsLoading(true);

  setError("");

  setSelectedAnswer("");

  setHint("");

  setShowClues(false);

  async function load() {
    try {
      const data = await getMysteryById(id);

      if (ignore) return;

      if (data.solved) {
        navigate(`/result/${data.id}`, { replace: true });

        return;
      }

      const clueData = await getClues(id);

      if (ignore) return;

      setMystery(data);

      setClues(clueData);

      setHintsRemaining(data.hintsRemaining);
    } catch (error) {
      if (!ignore) {
        setError(
          error instanceof Error ? error.message : "Could not load the game.",
        );
      }
    } finally {
      if (!ignore) setIsLoading(false);
    }
  }

  void load();
}
```

```
return () => {  
  ignore = true;  
};  
}, [id, reload, navigate]);
```

6. Code Walkthrough

The effect declaration

```
useEffect(() => { ... }, [reload]);
```

What it does: Runs the effect after render, and again when `reload` changes.

Why it exists: We want to fetch on mount and after retry.

What you see: The list appears; clicking retry re-fetches.

The ignore flag

```
let ignore = false;  
  
...  
  
if (!ignore) setMysteries(data);  
  
...  
  
return () => { ignore = true; };
```

What it does: Prevents state updates from a stale response.

Why it exists: If the component unmounts or the dependency changes, the old fetch should not touch state.

What you see: No React warnings about updating unmounted components.

In Plain English: "If this effect is no longer current, ignore the response."

Read it as: "Local flag, set on cleanup."

The async `load` function

```
async function load() { ... }  
  
void load();
```

What it does: Defines and runs the fetch.

Why it exists: `useEffect` cannot be async directly; the inner function handles that.

What you see: Nothing directly; the data appears when ready.

In Plain English: "Call the async function without awaiting it."

The try/catch/finally

```
try {  
  
  const data = await getMysteries();  
  
  if (!ignore) setMysteries(data);  
  
} catch (error) {  
  
  if (!ignore) setError( ... );  
  
} finally {  
  
  if (!ignore) setIsLoading(false);  
  
}
```

What it does: Handles success, failure, and always clears loading.

Why it exists: The page must leave the loading state no matter what.

What you see: Loading message disappears even on error.

7. Common Mistakes

WRONG: Setting state unconditionally after await.

```
const data = await getMysteries();

setMysteries(data); // may set state after unmount
```

CORRECT:

```
if (!ignore) setMysteries(data);
```

WRONG: Forgetting the cleanup return.

```
useEffect(() => {

  fetch(...).then(setData);

}, [id]);
```

Without cleanup, a stale response for the old id can overwrite the new one.

WRONG: Not clearing loading on error.

```
try {

  ...

  setIsLoading(false);

} catch (error) {

  setError(...); // isLoading stays true forever

}
```

Use `finally`.

8. Important Decisions

- **Why `ignore` instead of `AbortController`?** Simpler; the fetch cannot be truly cancelled anyway.
- **Why `finally`?** Loading state must always clear.

- **Why `void load()` ?** To make explicit that we are not awaiting.
- **Why the effect depends on `id` and `reload` ?** So it re-runs when either changes.

9. Project Flow

Component mounts

|

▼

Effect runs: `ignore = false`, `load()` called

|

▼

`await getMysteryById(id)`

|

▼

Component unmounts or `id` changes

|

▼

Cleanup: `ignore = true`

|

▼

Response arrives: `if (!ignore) → skipped`

10. Review Questions

1. What does `useEffect` do?
2. Why does the effect return a function?
3. What does the `ignore` flag prevent?

4. Why use `finally` ?
5. Why does the `GamePage` effect depend on `id` ?

11. Glossary

| Term | Meaning |
|------|---------|
|------|---------|

| Effect | Code that synchronizes with something outside rendering. |

| Cleanup | A function returned from `useEffect` that runs before the next effect. |

| Stale response | A response from an earlier fetch that arrives late. |

| Ignore flag | A local variable that blocks state updates after cleanup. |

12. What You Should Be Able to Explain Now

- What `useEffect` does and when it runs.
- Why cleanup matters.
- How the `ignore` flag works.
- Why `finally` clears loading.

13. Not Yet

- Game flow handlers — Session 3.

Session 3 — Game Flow: Submit, Hint, Reload

Learning Objectives

By the end of this session, the team should understand:

- How `handleSubmit` processes an answer.
- How `handleHint` requests a hint.
- How `reloadGame` recovers after an error.
- How the `busy` and `active` refs guard async behavior.

1. The Problem

The game has three actions: submit an answer, request a hint, and reload after an error. Each is an async operation. Without guards, the player could double-submit, and state could update after unmount.

2. Familiar Idea

In earlier sessions, we attached listeners to buttons with `addEventListener` and called functions on click.

The same idea applies here, but the handlers are async and need guards.

3. The New Concept

A **ref** is a mutable value that does not cause a re-render. The project uses two refs:

- **busy** — blocks overlapping answer/hint actions immediately.
- **active** — blocks state updates after unmount.

4. Tiny Example

```
function Form() {  
  const busy = useRef(false);  
  
  async function submit() {  
    if (busy.current) return;  
  
    busy.current = true;  
  
    try {  
      await save();  
    } finally {  
      busy.current = false;  
    }  
  }  
  
  return <button onClick={submit}>Save</button>;  
}
```

5. Project Application

File: `client/src/pages/GamePage.tsx`

```
const active = useRef(false);

const busy = useRef(false);

useEffect(() => {
  active.current = true;

  return () => {
    active.current = false;
  };
}, []);

async function handleSubmit(event: FormEvent<HTMLFormElement>) {
  event.preventDefault();

  if (!selectedAnswer.trim() || busy.current || isLoading || error || !mystery)
    return;

  busy.current = true;

  setPending("answer");

  setMessage("");

  try {
    const response = await submitAnswer(id, selectedAnswer.trim());

    if (!active.current) return;

    setMessage(response.message);

    setFeedbackTone(response.correct ? "success" : "wrong");

    if (response.correct) {
      setIsLoading(true);

      setReload((value) => value + 1);
    }
  } catch (error) {
    if (active.current) {
      setError(
        error instanceof Error ? error.message : "Could not submit the answer.",
      );
    }
  }
  finally {
    busy.current = false;
  }
}
```

```
    if (active.current) setPending(null);
  }
}

async function handleHint() {
  if (busy.current || isLoading || error || hintsRemaining === 0 || !mystery)
    return;

  busy.current = true;

  setPending("hint");

  setMessage("");

  try {
    const response = await requestHint(id);

    if (!active.current) return;

    setHint(response.hint);

    setHintsRemaining(response.hintsRemaining);
  } catch (error) {
    if (active.current) {
      setError(
        error instanceof Error ? error.message : "Could not request a hint.",
      );
    }
  } finally {
    busy.current = false;

    if (active.current) setPending(null);
  }
}

function reloadGame() {
  setMessage("");

  setReload((value) => value + 1);
}
```

6. Code Walkthrough

The `active` ref

```
const active = useRef(false);

useEffect(() => {
  active.current = true;

  return () => {
    active.current = false;
  };
}, []);
```

What it does: Tracks whether the component is still mounted.

Why it exists: Async handlers should not update state after unmount.

What you see: No "can't perform a React state update on an unmounted component" warnings.

The `busy` ref

```
const busy = useRef(false);
```

What it does: Blocks overlapping submissions immediately.

Why it exists: `pending` state updates asynchronously; `busy.current` updates synchronously.

What you see: Double-clicking Submit does not send two requests.

handleSubmit

```
async function handleSubmit(event) {

  event.preventDefault();

  if (!selectedAnswer.trim() || busy.current || isLoading || error || !mystery)
    return;

  busy.current = true;

  setPending("answer");

  setMessage("");

  try {

    const response = await submitAnswer(id, selectedAnswer.trim());

    if (!active.current) return;

    setMessage(response.message);

    setFeedbackTone(response.correct ? "success" : "wrong");

    if (response.correct) {

      setIsLoading(true);

      setReload((value) => value + 1);

    }

  } catch (error) {

    if (active.current) setError( ... );

  } finally {

    busy.current = false;

    if (active.current) setPending(null);

  }

}
```

```
}
```

What it does: Submits the answer, shows feedback, and on correct, re-fetches.

Why it exists: The main action of the game.

What you see: "Correct!" or "Wrong answer. Try again." then the next stage.

In Plain English: "Prevent default, guard, mark busy, POST, update state."

Why re-fetch after correct

```
setIsLoading(true);  
  
setReload((value) => value + 1);
```

What it does: Triggers the load effect again.

Why it exists: The answer response is not a complete mystery. The next question, hint budget, and solved flag must come from the server.

What you see: After a correct answer, the next stage appears.

handleHint

```
async function handleHint() {

  if (busy.current || isLoading || error || hintsRemaining === 0 || !mystery)
    return;

  busy.current = true;

  setPending("hint");

  setMessage("");

  try {

    const response = await requestHint(id);

    if (!active.current) return;

    setHint(response.hint);

    setHintsRemaining(response.hintsRemaining);

  } catch (error) {

    if (active.current) setError( ... );

  } finally {

    busy.current = false;

    if (active.current) setPending(null);

  }

}
```

What it does: Requests a hint and updates the budget.

Why it exists: The player may need help.

What you see: The hint appears; the remaining count drops.

reloadGame

```
function reloadGame() {  
  setMessage("");  
  
  setReload((value) => value + 1);  
}
```

What it does: Clears feedback and re-fetches.

Why it exists: After an error, the safest action is to read current server state before trying again.

What you see: The page reloads its data.

In Plain English: "Read current progress instead of blindly retrying."

7. Common Mistakes

WRONG: Relying only on `pending` state to block overlap.

State updates are asynchronous. Two rapid clicks can both pass the guard.

CORRECT: Use `busy.current`, which updates synchronously.

WRONG: Not checking `active.current` before setting state.

The component may have unmounted.

WRONG: Resending the mutation instead of re-reading.

If the server accepted the answer but the response was lost, resending could double-apply. Re-reading is safer.

8. Important Decisions

- **Why `busy` and `active` refs?** Immediate synchronous guards.
- **Why re-fetch after correct?** The server owns the next question and progress.
- **Why `reloadGame` reads instead of resubmits?** Safe recovery from partial failures.
- **Why `event.preventDefault()` ?** Without it, the form submits and reloads the page.

9. Project Flow

Player selects "brass" → selectedAnswer state updates

|

▼

Player clicks Submit

|

▼

handleSubmit: preventDefault, guard, busy = true, pending = "answer"

|

▼

POST /api/mysteries/1/answers

|

▼

Server: correct → nextStage = 1

|

▼

handleSubmit: setMessage("The lock clicks open ... "), setReload+1

|

▼

Effect re-runs, fetches updated mystery

|

▼

Next stage renders

10. Review Questions

1. What does `event.preventDefault()` prevent?
2. Why does the project use a `busy` ref instead of only `pending` state?
3. Why does the component re-fetch after a correct answer?
4. Why does `reloadGame` read instead of resubmit?
5. What does `active.current` guard against?

11. Glossary

| Term | Meaning |
|------|---------|
|------|---------|

| Ref | A mutable value that does not cause a re-render. |

| Guard | A check that returns early if preconditions are not met. |

| Overlap | Two async actions running at the same time. |

| Re-fetch | Reading fresh data from the server. |

| Partial failure | A server change that occurred but whose response was lost. |

12. What You Should Be Able to Explain Now

- How `handleSubmit`, `handleHint`, and `reloadGame` work.
- Why the project uses `busy` and `active` refs.
- Why re-fetching is safer than resubmitting.
- Why `preventDefault` is necessary.

13. Not Yet

- The visual game components — that is Adham's session.
- The server-side validation and status codes — that is Hamid's session.